COMP4330 / P01 GROUP 2 / FINAL COMBINED REPORT

The same predictions. Less waiting.

Understanding scikit-learn, ONNX and the results of our CPU experiments

An English guide for readers with no machine-learning background
4 November 2026 | Evidence collected 14 October - 4 November 2026

15	180,000	114
unique experiment runs	timed prediction calls	distinct test cases

The question

Can the same trained model make its predictions faster after conversion to ONNX, while keeping the same tested classification decisions?

What we found

ONNX Runtime completed the tested prediction calls faster than scikit-learn on both Windows computers. Converted models agreed with their scikit-learn originals on all 114 test cases. The amount of acceleration changed with the computer, number of trees, batch size and output format.

The 100-tree model correctly classified **109 of 114 cases (95.61%)**. Agreement between software tools was 100%; prediction accuracy was not.

How to read this report

Pages 2-5: learn the concepts and understand the experiment.
Pages 6-12: explore the results and their practical meaning.
Pages 13-16: read the conclusions, detailed tables and evidence guide.

The two input reports overlap. Their results are consolidated here once; repeated copies and repeated descriptions are not counted as new experiments.

Source: [R1], pp. 1-3, 12-17; [R2], pp. 1-8. Reference keys appear on page 16.

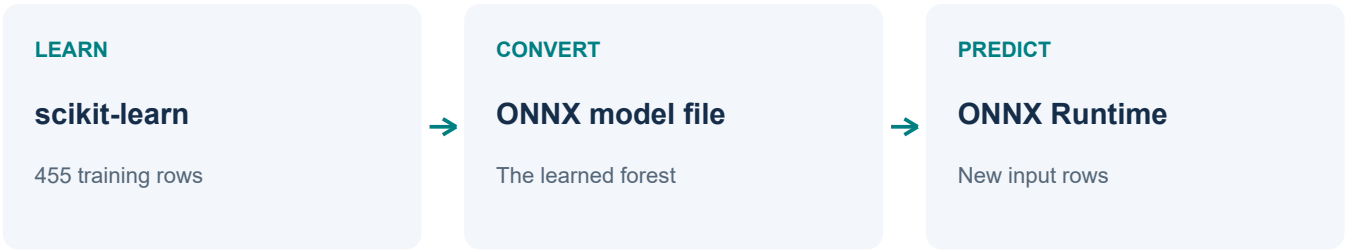
01 / START WITH THE BASICS

What are we comparing?

A machine-learning model learns a pattern from examples. Once trained, it takes a new input and produces a prediction. Using the trained model is called **inference**.

Name	Plain-English meaning	Role in this project
scikit-learn	A Python library: a collection of reusable machine-learning tools.	Trains the forest and provides the original prediction API.
ONNX	Open Neural Network Exchange: a standard model representation.	Stores the trained forest in a format a compatible runtime can execute.
ONNX Runtime (ORT)	Software that executes models saved in ONNX format.	Runs the converted forest on the CPU.

Python is a programming language. An API is a software call used by an application. A CPU is the computer's general-purpose processor.



The conversion step changes how the learned model is represented and executed. It does not train a new forest. We compare the original prediction path with two ONNX output formats.

What is a Random Forest?

A decision tree follows a sequence of learned questions about input values. A **Random Forest** combines the probabilities produced by many trees. This study uses forests with 10, 100 or 500 trees. Changing the number of trees creates a different model.

What goes in, and what comes out?

Each input row contains **30 numerical measurements** from a breast-cancer diagnostic dataset. The model produces probabilities for two classes, malignant (cancerous) and benign (non-cancerous), and a class label. These are numerical records, not medical images.

Keep three ideas separate

Training learns a model. **Conversion** saves that model in another representation. **Inference** uses it to predict. This report primarily measures inference.

The dataset is used for a software performance comparison; this experiment does not establish clinical suitability.

Source: [R1], pp. 2-3; [R2], pp. 1, 8. Diagram is an explanatory illustration.

02 / READ THE NUMBERS

Speed has several meanings

A **batch** is the number of input rows processed in one prediction call. A batch of 16 means one call handles 16 rows together.

Measure	What it tells you	Preferred direction
Latency	How long one complete batch call takes.	Lower
Throughput	How many rows per second the measured call rate implies.	Higher
Speedup	scikit-learn latency divided by ONNX latency for the same run and batch.	Above 1x means ONNX is faster
Median / P50	The middle timing when observations are sorted.	Describes a typical call
P95	A timing threshold covering about 95% of observed calls.	Shows slower calls as well

A worked example (illustrative numbers)

Suppose a batch of 16 takes **2.00 ms** in scikit-learn and **0.02 ms** in ONNX Runtime.
Speedup = 2.00 / 0.02 = 100x.
Throughput = 16 / 0.00002 seconds = 800,000 rows/second.
One millisecond (ms) is one thousandth of a second.

What is actually inside the stopwatch?

The stopwatch surrounds the **probability prediction API**, including creation of its returned objects. Models and runtime sessions are already prepared, and 50 warm-up calls have already run.

It excludes model loading, session creation, input preparation, later output conversion/validation and destruction of returned objects. It also excludes a real service's request transport, queueing and application processing.

Read the large speedup numbers in context

A 100x faster prediction call does not imply a whole website or service is 100x faster. Similarly, the reported rows/second are derived from API latency, not measured under live user traffic.

P25-P75 describes the middle 50% of timings. Its width is the interquartile range (IQR). A three-run minimum-to-maximum range describes observed variation; it is not a confidence interval.

Source: [R1], pp. 2, 6-7; [R2], pp. 1-2, 8. Worked example is invented for explanation.

03 / EXPERIMENTAL DESIGN

One dataset, two connected studies

The Breast Cancer Wisconsin Diagnostic dataset contains 569 records. A fixed, stratified 80/20 split keeps similar class proportions in the training and test sets; seed 42 makes the split repeatable.

Dataset part	Malignant	Benign	Total
Training: used to learn	170	285	455
Test: held out from training	42	72	114
All records	212	357	569

Training and test indices do not overlap. Inputs use contiguous float32 arrays (a fixed numerical storage format). Accuracy is measured only on the 114 original test records.

Study	Unique runs and purpose	Timed calls
A: same 100-tree model	RUN-001: AMD original; RUN-002: AMD workflow check; RUN-003: Intel reproduction; RUN-004 to 006: Intel controlled repeats.	6 x 12,000 = 72,000
B: tree-count study	AMD only: 10, 100 and 500 trees, each repeated three times. New code snapshot; analysed separately.	9 x 12,000 = 108,000

Each run: 3 prediction paths x 4 batches (1, 16, 64, 256). Each path/batch receives 50 warm-ups, then 10 rounds of 100 timed calls: **1,000 timings per configuration**. Runtime order rotates between rounds; Study B also rotates tree-count order.

Environment	AMD local computer	Intel computer
CPU	Ryzen AI 7 H 350	Core i7-12700
Physical / logical cores	8 / 16	12 / 20
Windows / Python	Windows 11 / 3.12.10	Windows 10 / 3.12.2

Both use scikit-learn 1.9.1, ONNX Runtime 1.31.0, skl2onnx 1.20.0 and NumPy 2.5.4. CPU execution is restricted to one thread in the model, native thread pools and ORT settings.

Important: batch 256 repeats the same 114 test cases to create a larger performance workload. Neither 180,000 timings nor repeated runs create additional independent accuracy cases.

Source: [R1], pp. 1-3, 12, 17; [R2], p. 1; [D1-D7]. The planned cloud VM was replaced by these two physical Windows computers.

04 / CORRECTNESS FIRST

Did conversion preserve the answers?

Accuracy compares predictions with the true dataset labels. **Agreement** compares ONNX predictions with scikit-learn predictions. A conversion can preserve the same mistakes.

Forest	Correct test cases	Accuracy	ONNX label agreement
10 trees	107 / 114	93.86%	100%
100 trees	109 / 114	95.61%	100%
500 trees	109 / 114	95.61%	100%

What 100% agreement means

On the tested cases, the original and converted versions chose the same class. For the 100-tree forest, they were all correct on 109 cases and all wrong on the remaining 5.

Do the probability values match too?

The class decisions match exactly, while probability values can differ slightly because numerical computation is not identical. The largest observed absolute differences were:

Forest	Largest probability difference	Required tolerance
10 trees	0.000000119209	Below 0.00001
100 trees	0.000000655651	Below 0.00001
500 trees	0.00000578165	Below 0.00001

All saved correctness checks passed for both ONNX output formats. The checks also cover output dimensions, class order, valid finite probabilities, values between 0 and 1, and row sums close to 1.

What does changing the tree count achieve?

Moving from 10 to 100 trees increased the total number of correct predictions by two. At 500 trees, the total remained 109. That makes 100 trees a reasonable trade-off on this fixed test set, but it does not establish a universally best forest size.

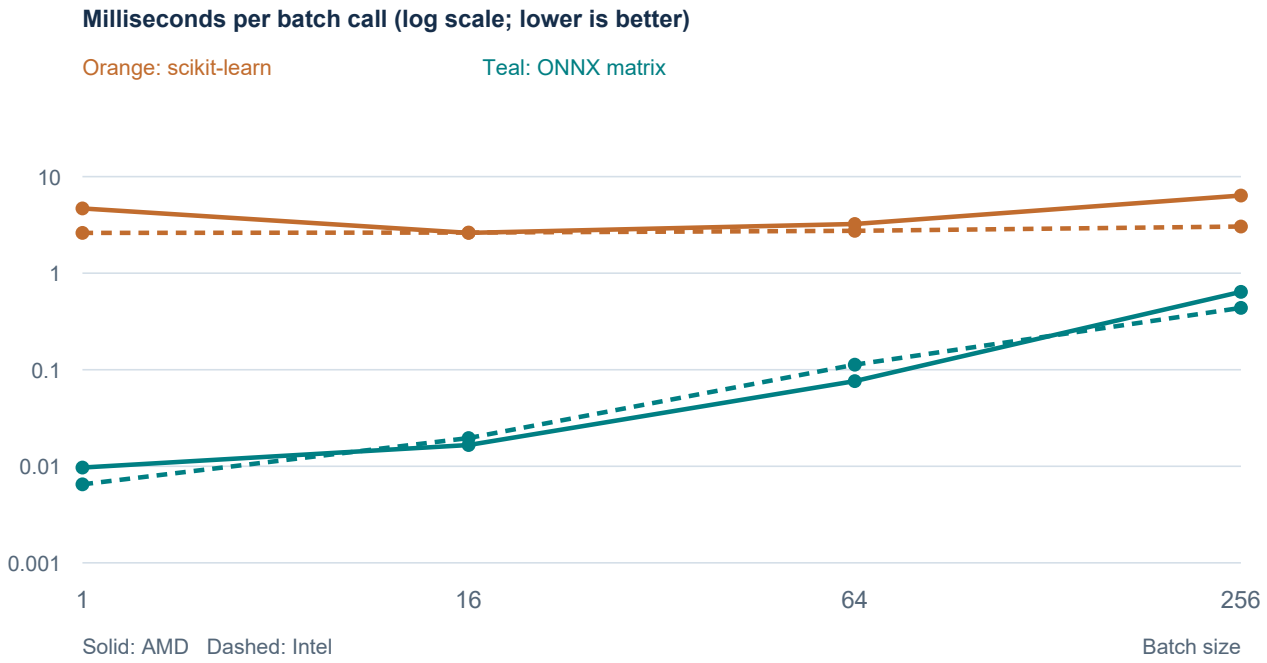
The three repeats use the same seed and test cases. They measure execution repeatability, not accuracy across three independent populations or data splits.

Source: [R1], pp. 3, 12; [R2], p. 8; saved correctness.json in [D1-D7].

05 / STUDY A: TWO COMPUTERS

ONNX was faster on both computers

This comparison holds the 100-tree model fixed. The solid lines use AMD RUN-001; dashed lines use the median of Intel RUN-004 to 006. ONNX matrix means ZipMap off (explained on page 10).



Equal vertical spacing means a tenfold change on this logarithmic axis. Intel points combine three run medians; the chart does not display timing or repeat ranges.

Batch	AMD ONNX ms	AMD speedup	Intel ONNX ms range	Intel speedup range
1	0.00970	482.93x	0.00650 - 0.00660	393.40 - 402.05x
16	0.01660	157.48x	0.01960 - 0.01970	133.04 - 134.94x
64	0.07620	42.41x	0.11070 - 0.11395	24.34 - 24.79x
256	0.64045	9.95x	0.43570 - 0.44090	6.95 - 6.98x

Example: AMD batch 1 fell from 4.6844 ms in scikit-learn to 0.00970 ms in ONNX: about 482.93x faster. Intel ONNX was faster still at 0.00650-0.00660 ms, but its relative speedup was smaller because its scikit-learn baseline was also faster.

A useful comparison, with a clear boundary

Intel ONNX latency was lower at batches 1 and 256, and higher at 16 and 64. CPU, operating system, Python patch version and operating conditions differ together, so these results do not isolate a CPU-only effect.

Source: [R1], pp. 4, 9-11; [D1], [D4-D6]. Ranges are observed three-run minima and maxima.

06 / REPEATABILITY

A typical time is only part of the story

RUN-004 to 006 are the three controlled short-term repeats on the Intel computer. RUN-003 is the first independent reproduction and is kept separate from these repeatability statistics.

Batch	RUN-004 median ms	RUN-005 median ms	RUN-006 median ms	P95 range ms
1	0.00650	0.00660	0.00650	0.00690 - 0.00720
16	0.01960	0.01960	0.01970	0.02130 - 0.02280
64	0.11280	0.11070	0.11395	0.11730 - 0.12311
256	0.43710	0.43570	0.44090	0.44780 - 0.46670

The ONNX matrix medians changed little between these repeats. This supports short-term repeatability on that computer. It does not prove the same stability over different days, machines or workloads.

Why keep P95?

For RUN-006, batch 1 with ZipMap on, the median was **0.0078 ms**, but P95 was **0.022205 ms**. The corresponding P95 in the first two repeats was about 0.0083-0.0085 ms. A stable median can hide a change in slower calls.

What was controlled?

The Intel operator recorded plugged-in power, Best performance mode and an idle computer. Successful-run logs record 60-second gaps. Each run used the same model, test data, batches and single-thread runtime settings.

What was not fully controlled?

The program did not fix CPU affinity (which core the operating system selects), frequency, temperature or Python garbage collection. AMD has no matching set of three controlled historical repeats, so the two machines cannot be ranked fairly for general stability.

One recorded launch stopped before measurement because a command-line option used lowercase "yes" where "Yes" was required. The successful restart produced the valid runs. This startup failure contributes no timing samples.

Practical lesson

For an application with a response-time target, measure both typical and slow calls under the intended workload. Three warm, fixed-input repeats are useful evidence, but do not establish a live-service response guarantee.

Source: [R1], pp. 6-7; [D4-D6]. P95 is computed within each run before reporting the observed range.

07 / STUDY B: NUMBER OF TREES

More trees change the benefit

The second study uses the AMD computer and compares 10, 100 and 500 trees. Each cell below is the median of three run-specific speedup ratios for ONNX matrix output.

ONNX matrix speedup relative to scikit-learn

	10 trees	100 trees	500 trees
Batch 1	54.50x	346.37x	1,051.97x
Batch 16	46.21x	131.74x	153.72x
Batch 64	27.07x	55.09x	24.35x
Batch 256	13.76x	9.95x	6.96x

The darker cells indicate larger speedups. Each ratio compares the same model and batch within the same run; it is not a ratio between different forest sizes.

Look at relative and absolute time together

Forest	Batch 1: sklearn / ONNX ms	Batch 256: sklearn / ONNX ms
10 trees	0.60500 / 0.01110	0.66125 / 0.04690
100 trees	4.46815 / 0.01290	5.06020 / 0.50160
500 trees	20.98095 / 0.02000	24.53225 / 3.52200

At batch 1, the relative speedup increases from 54.50x to 1,051.97x as the forest grows. Yet ONNX itself takes longer for 500 trees than for 10 trees: 0.0200 ms versus 0.0111 ms. A bigger speedup ratio does not mean a bigger model is faster.

At batch 256, the relative speedup falls from 13.76x to 6.96x as the forest grows. ONNX still remains faster than scikit-learn in the tested comparisons.

What the largest number means

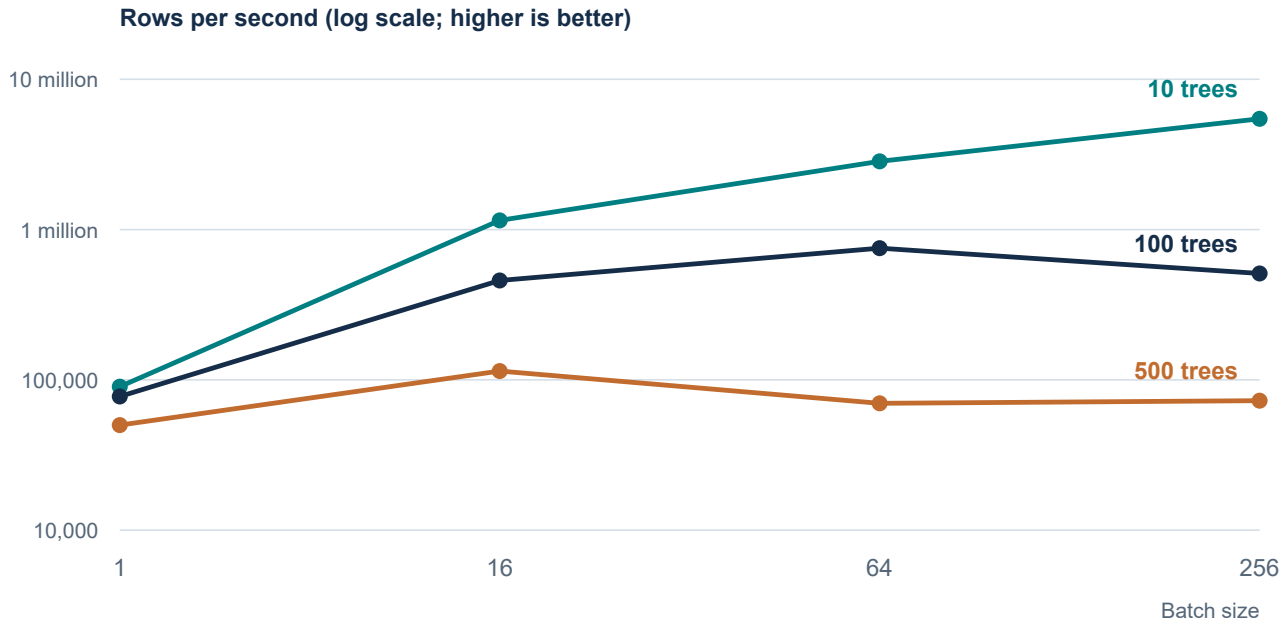
The 1,051.97x result is a warm probability-API observation for 500 trees and batch 1 on this AMD setup. It describes one workload, rather than a general acceleration factor for ONNX.

Source: [R1], p. 12; [R2], pp. 2-4; [D7] summary.csv. Separate medians need not divide to exactly reproduce the median speedup.

08 / BATCH SIZE

A larger batch is not always better

Processing rows together can share per-call work. But the best batch depends on model size and on whether the application values throughput or the waiting time of an individual request.



Study B, ONNX matrix output. Points show the median of three run-specific throughput values. Lines connect the four tested batch sizes; intermediate batches were not tested.

Forest	Best tested batch	Rows/second	Batch-call latency ms
10 trees	256	5,458,422	0.0469
100 trees	64	752,056	0.0851
500 trees	16	114,449	0.1398

These are the best of four tested choices, not a search over every possible batch. In the earlier 100-tree study, AMD RUN-001 and the Intel repeats peaked at batch 16; AMD RUN-002 peaked at 64. A fixed rule such as "always use batch 16" would miss this variation.

Throughput and user waiting time are different

A queue may need to wait for enough requests to form a batch. That waiting time is outside this benchmark. Choose batches using the application's arrival rate, latency target and real input variation.

All throughput figures are derived from repeated fixed-input API calls. The batch-256 workload contains repeated test records; these millions of rows/second are not millions of newly evaluated cases.

Source: [R1], pp. 3, 5, 12; [R2], pp. 1-2; [D1-D7].

09 / OUTPUT FORMAT

What is ZipMap, and why test it?

The prediction contains a probability for each class. ONNX can return those numbers as labelled maps or as a rectangular numerical array. The values serve the same purpose, but the output objects differ.

ZipMap on: labelled maps	ZipMap off: probability matrix
One map per row, with class identifiers attached to probabilities. Example: {0: 0.20, 1: 0.80}	One row per input, with a fixed column for each class. Example: [0.20, 0.80]
Convenient if the next step expects maps.	Convenient if the next step processes arrays. Often called a tensor.

Examples are invented. Column order must be known: class 0 is malignant and class 1 is benign in this dataset. The validation checks class order.

Observed latency reduction from turning ZipMap off

Forest	Batch 1	Batch 16	Batch 64	Batch 256
10 trees	14.62%	26.46%	45.61%	60.93%
100 trees	14.57%	14.67%	33.65%	15.20%
500 trees	16.94%	5.16%	2.76%	2.96%

Study B values are medians of three paired reductions: 100 x (1 - off latency / on latency). Turning ZipMap off was faster in **35 of 36** model/run/batch pairs. All 24 pairs in Study A also favoured off.

The exception matters

For 500 trees, repeat 1, batch 1: ZipMap on took **0.01755 ms**; off took **0.02000 ms**. Turning it off increased latency by **13.96%**. The three-run reduction range for this setting was -13.96% to 21.67%.

Deployment implication

Test matrix output first when the application can use it directly. If the application needs maps, include any required conversion in the comparison. The measured benefit combines graph and output-representation changes; it cannot all be assigned to one operator.

Source: [R1], pp. 5, 13; [R2], pp. 2, 7. Paired reductions independently recalculated from [D7] runs.csv.

10 / EXPLAINING THE RESULT

Why might the execution paths differ?

The model's learned decisions are preserved, but the software takes a different route to calculate them. Profiling means recording where execution work occurs to help investigate that route.

scikit-learn prediction path	ONNX Runtime prediction path
Python-level input checks, per-tree dispatch, probability accumulation and averaging.	The forest is encoded as a TreeEnsembleClassifier operator and executed by a native ensemble kernel.
Individual tree traversal already uses compiled code.	ZipMap-on graphs also show ZipMap and Cast execution.

What the profiles actually showed

For 500 trees and batch 1, a 100-call scikit-learn profile recorded **50,000 per-tree dispatch-wrapper calls** and **50,000 tree probability calls**. This matches 500 trees being visited for each of 100 forest predictions.

Repeated warning-filter setup was also visible; that records warning-handling operations, not evidence that thousands of warnings were emitted. ORT traces showed the forest kernel and, for map output, the additional output operators.

Evidence for a mechanism, not an exact cost breakdown

The profiles are consistent with less Python-level per-tree orchestration in the ONNX path. They do not tell us exactly what fraction of the benchmark speedup each internal step caused.

Why the profiler times cannot simply be subtracted

Profiling was performed after formal benchmarking and adds its own overhead. Python cProfile and ORT kernel traces observe different boundaries. Some compiled work is attributed to its Python caller, and ORT kernel times omit parts of the Python interface and returned-object creation.

Therefore, subtracting a profiled kernel time from an unprofiled API latency does not produce a precise "Python overhead" value. Nor does a smaller ONNX file prove that cache behaviour or vector instructions caused the speedup.

Source: [R1], p. 13; [R2], pp. 5-8. This is an interpretation of saved profiles, not a new profiling run.

11 / DEPLOYMENT TRADE-OFFS

Prediction time is not the only cost

A deployment also has to store the model and prepare it for use. The tree study measured these steps separately from warm prediction timing.

Forest	Tree nodes	Max depth	sklearn KiB	ONNX matrix KiB
10 trees	326	10	30.63	12.25
100 trees	3,556	10	316.73	127.61
500 trees	17,754	11	1,575.95	654.11

A node is one decision or end point in a tree. Depth describes the longest decision path. KiB means 1,024 bytes. These are saved-file sizes, not runtime memory use.

Preparation costs in Study B

Step (milliseconds)	10 trees	100 trees	500 trees
Train forest	10.823	141.425	672.133
Load sklearn forest	8.018	17.502	79.661
Convert to ONNX matrix	12.669	140.846	610.695
Create ORT matrix session	0.740	2.583	12.574

Each phase was observed once per child run; the table reports the median of three observations. Training excludes saving. Conversion excludes model checking and serialization. These are not full cold-start distributions.

Pay costs at the right point in the lifecycle

Training and conversion normally prepare the model before deployment. Loading and session creation happen when a process prepares the model for use. They are outside the warm-call speedup and should not be charged to every request.

A balanced choice for this experiment

The 100-tree forest achieved the same held-out accuracy as 500 trees with a smaller file and lower ONNX latency. The 10-tree forest was smaller and faster but correctly classified two fewer test cases. The preferred trade-off depends on the application.

Do not add these phase medians to claim a complete cold-start time: the phases exclude some work and represent different lifecycle operations.

Source: [R1], pp. 14-16; [R2], pp. 5, 8; [D7] model_sizes.csv and setup_times.csv.

12 / CONCLUSIONS

What should a reader take away?

ONNX Runtime accelerated the tested probability prediction calls while preserving the tested class decisions. This is evidence for a useful deployment option, with benefits that depend on the workload.

Finding	Practical meaning
Correctness was preserved on the test set.	Conversion changed execution without changing any of the 114 tested class decisions. It did not improve accuracy.
Warm prediction calls became faster.	Measure the full application before translating API speedup into a service claim.
Tree count and batch interact.	Test the target forest and batches; bigger batches or models do not guarantee a better trade-off.
Matrix output usually helped.	Prefer testing it when downstream code uses arrays; keep the measured exception and conversion costs in view.

What the evidence does not establish

The experiment uses one dataset, one fixed train/test split and only two Windows computers. The tree-count extension uses the AMD computer only. Repeated fixed inputs and three short-term reruns do not represent arbitrary new populations, varying inputs or production traffic.

The machine comparison changes CPU, Windows and Python version together. Some power/load conditions were recorded manually, while frequency, temperature, core placement and garbage collection were not fixed by the program. Ranges are descriptive, not statistical confidence intervals.

Recommended next validation

1. Keep correctness checks when converting a model or changing output format.
2. Benchmark realistic inputs, batches and downstream output handling.
3. Measure loading, cold starts, queueing, concurrent requests and slow-call percentiles in the full application.
4. For a hardware claim, repeat both machines under matching software and operating conditions.

A defensible final conclusion

For the tested Random Forests, ONNX Runtime offered substantially faster warm CPU prediction calls with unchanged tested classifications. The best deployment configuration must be selected using the intended model, output format and workload.

This report combines and explains saved evidence. No models were trained and no new inference, benchmark, profiling or test-suite runs were performed to prepare it.

Source: [R1], pp. 6-8, 12-17; [R2], pp. 1-8. Recommendations are proposed next steps, not completed experiments.

APPENDIX A / STUDY A REFERENCE TABLE

The six original 100-tree runs

Each latency cell is median / P95 in milliseconds per complete batch call. Each row draws on 1,000 timings per prediction path. M = ONNX maps (ZipMap on); T = ONNX matrix (ZipMap off).

RUN	Batch	sklearn median / P95	M median / P95	T median / P95	T speedup
001	1	4.68440 / 12.19507	0.01075 / 0.04113	0.00970 / 0.03623	482.93x
001	16	2.61425 / 4.10991	0.01960 / 0.03970	0.01660 / 0.02760	157.48x
001	64	3.23200 / 5.21484	0.08960 / 0.17732	0.07620 / 0.10982	42.41x
001	256	6.37035 / 11.54086	0.70925 / 1.59343	0.64045 / 1.39473	9.95x
002	1	4.60510 / 7.29590	0.01305 / 0.02021	0.01280 / 0.01550	359.77x
002	16	4.38445 / 7.45182	0.04380 / 0.05634	0.03520 / 0.04925	124.56x
002	64	4.76555 / 8.61608	0.09100 / 0.17464	0.07780 / 0.23148	61.25x
002	256	5.16000 / 6.93837	0.60490 / 0.90222	0.48845 / 0.71652	10.56x
003	1	2.56825 / 2.65225	0.00770 / 0.00840	0.00650 / 0.00700	395.12x
003	16	2.62860 / 2.77011	0.02390 / 0.02710	0.01990 / 0.02210	132.09x
003	64	2.76105 / 2.88981	0.12380 / 0.13140	0.11130 / 0.11900	24.81x
003	256	3.03210 / 3.12231	0.48570 / 0.49950	0.43750 / 0.44890	6.93x
004	1	2.55710 / 2.73554	0.00770 / 0.00830	0.00650 / 0.00690	393.40x
004	16	2.60750 / 2.70881	0.02370 / 0.02710	0.01960 / 0.02240	133.04x
004	64	2.74590 / 2.88048	0.12630 / 0.13550	0.11280 / 0.12100	24.34x
004	256	3.03575 / 3.13236	0.48620 / 0.50960	0.43710 / 0.46670	6.95x
005	1	2.61920 / 2.76378	0.00790 / 0.00850	0.00660 / 0.00720	396.85x
005	16	2.63425 / 2.73965	0.02360 / 0.02811	0.01960 / 0.02130	134.40x
005	64	2.74390 / 2.92341	0.12370 / 0.13090	0.11070 / 0.11730	24.79x
005	256	3.04255 / 3.18262	0.48550 / 0.50831	0.43570 / 0.44780	6.98x
006	1	2.61335 / 2.76387	0.00780 / 0.02220	0.00650 / 0.00700	402.05x
006	16	2.65825 / 2.77633	0.02390 / 0.02760	0.01970 / 0.02280	134.94x
006	64	2.79545 / 2.94241	0.12630 / 0.13671	0.11395 / 0.12311	24.53x
006	256	3.07875 / 3.26306	0.48820 / 0.50201	0.44090 / 0.45391	6.98x

Run roles: 001 AMD original (14 Oct); 002 AMD workflow validation (30 Oct); 003 Intel reproduction (3 Nov); 004-006 Intel controlled repeats (3 Nov). The six runs are not pooled for repeatability statistics.

Speedup uses the corresponding scikit-learn median from the same run and batch. Complete P25/P75 and per-call observations remain in each run's summary.csv and timings.csv.

Source: [R1], pp. 9-11; [D1-D6]. Values directly loaded from the six saved summary.csv files.

APPENDIX B / STUDY B REFERENCE TABLE

Tree-count results at a glance

Each latency cell is median / P95 in milliseconds. Each statistic is calculated within each of three runs, then its median is taken. M = maps; T = matrix output.

Trees	Batch	sklearn median / P95	M median / P95	T median / P95	T speedup [range]
10	1	0.60500 / 0.93984	0.01300 / 0.01520	0.01110 / 0.01330	54.50x [42.90, 70.00]
10	16	0.61420 / 0.99494	0.01890 / 0.02820	0.01390 / 0.02100	46.21x [40.41, 54.54]
10	64	0.60905 / 0.90493	0.04200 / 0.05342	0.02250 / 0.02835	27.07x [23.89, 37.65]
10	256	0.66125 / 1.03002	0.12740 / 0.15932	0.04690 / 0.07679	13.76x [12.00, 17.97]
100	1	4.46815 / 7.55619	0.01480 / 0.01990	0.01290 / 0.02070	346.37x [345.79, 457.58]
100	16	4.50520 / 6.78021	0.04060 / 0.04890	0.03490 / 0.04875	131.74x [127.99, 135.51]
100	64	4.70930 / 6.57597	0.13270 / 0.26390	0.08510 / 0.16355	55.09x [41.79, 59.05]
100	256	5.06020 / 7.57870	0.59315 / 1.03719	0.50160 / 0.65449	9.95x [9.63, 10.86]
500	1	20.98095 / 30.07697	0.02400 / 0.02970	0.02000 / 0.02510	1051.97x [1042.61, 1116.01]
500	16	21.43275 / 30.15644	0.14535 / 0.27521	0.13980 / 0.22304	153.72x [148.63, 157.37]
500	64	22.42995 / 31.91671	0.94140 / 1.20075	0.91680 / 1.47132	24.35x [23.41, 24.86]
500	256	24.53225 / 34.18291	3.62945 / 5.27197	3.52200 / 4.86334	6.96x [6.88, 6.97]

How these summaries were calculated

Each run supplies 1,000 calls per configuration. We report the median of three run-level statistics, so P95 is **not** calculated from 3,000 pooled calls. Dividing the displayed latency medians may not reproduce the median speedup.

Square brackets contain the minimum and maximum of three speedup ratios. They are observed ranges, not confidence intervals. All three repeats use the same fixed dataset split and model seed.

Additional setup observations

Step, ms (three-observation median)	10 trees	100 trees	500 trees
ONNX maps conversion	36.933	163.038	648.787
ORT maps session creation	1.382	3.097	15.291

Map-format files were 12.40, 127.75 and 654.25 KiB. Setup observations are separate from prediction timing.

Source: [R2], pp. 2, 5, 8; [D7] summary.csv, model_sizes.csv and setup_times.csv.

APPENDIX C / GLOSSARY AND EVIDENCE GUIDE

Terms, sources and verification

Term	Meaning in this report
API / prediction path	The software call an application uses to obtain predictions.
Backend / runtime	The software implementation that executes the model.
Feature / label	A numerical model input / the true or predicted class.
Tensor / kernel	A numerical array / native code executing a model operator.
Warm-up / session	Untimed preparation calls / a prepared ORT model execution object.
Seed / stratified split	A repeatable random setting / a split preserving class proportions.
Hash / provenance	A file-content fingerprint / records connecting results to their inputs and code.

Primary reports

[R1] ONNX_Local_vs_New_Results_20261104.pdf, 17 pages.
[R2] ONNX_Tree_Study_Results_20261104.pdf, 8 pages.
Both are in output/pdf/. R1 already includes Study B; R2 presents that study in more detail. This English report reorganizes their overlapping evidence.

Saved result locations (relative to the project root)

[D1] output/experiment
[D2] output/runs/20261030T154052Z_be01f368
For D3-D6, B = new/COMP4330_Results_MC235E10_20261103_114239_UTC/output/runs/
[D3] B/20261103T113724Z_5d3a5108
[D4] B/20261103T113908Z_8f07f50f
[D5] B/20261103T114026Z_2feb004b
[D6] B/20261103T114144Z_9993ceea
[D7] output/tree_study_20261104, including repeat_1 to repeat_3.

What was checked for this English edition?

The authoring check reread 180,000 raw timings across 15 unique runs, confirmed 1,000 calls for each of 180 configurations, and recalculated all 180 within-run latency medians. Saved correctness records were checked for PASS, 114 test cases and 100% label agreement. English figures and tables use saved CSV values.

Earlier integrity audits are reported in R1, pp. 8, 13 and 17: manifests, artifact/source hashes, model structure, workbook reconciliation and profile parsing. These historical checks and the saved 38-test/mypy result for the tree study were not rerun for this edition.

Code snapshots are distinct: Study B added profiling/study modules and changed existing modules. The 100-tree forest matches the earlier forest, but the full current code is not identical to the original snapshot. Use each run's provenance records.

Numbers are rounded for readability. Dates in the narrative use Hong Kong local dates; some saved run identifiers use UTC. No new experimental observations were added.

Source: The two supplied reports plus the directly listed saved result files. Source PDFs remain unchanged.